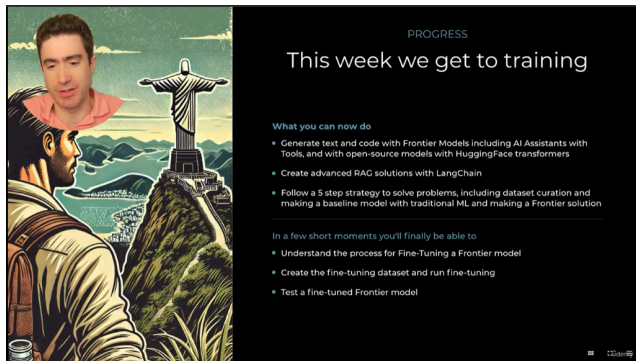


02 Tháng Tư 2025 10:54 CH



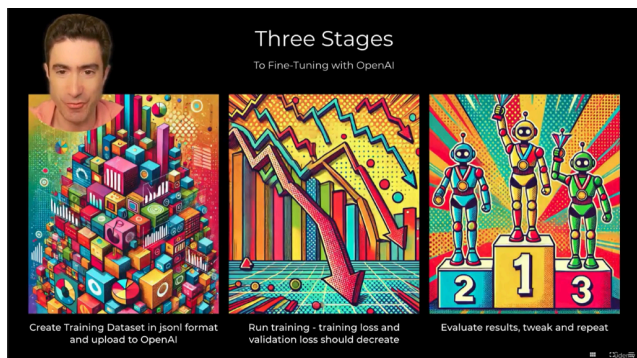
Tuần này tập trung vào **fine-tuning mô hình frontier**, tức là điều chỉnh một mô hình đã được huấn luyện để cải thiện hiệu suất cho một tác vụ cụ thể.

Đã sử dụng **các mô hình frontier** để tạo văn bản và code.
 Đã làm việc với **Hugging Face**, dùng cả **pipelines** và API transformers.
 Đã tạo **RAG pipelines** (truy xuất và tạo văn bản) có và không có **LangChain**.
 Đã áp dụng **chiến lược 5 bước giải quyết vấn đề**, trong đó một phần quan trọng là **chuẩn bị dữ liệu**.
 Đã huấn luyện mô hình truyền thống (baseline model), nhưng **chưa thực hiện fine-tuning mô hình frontier**.

Hiệu quy trình fine-tuning cho mô hình frontier.
Tạo tập dữ liệu huấn luyện phù hợp.
Chạy quá trình fine-tuning.
Kiểm tra mô hình sau khi fine-tuning để đánh giá sự cải thiện.

Fine-tuning = đào tạo bổ sung trên mô hình có sẵn, tận dụng **transfer learning**.
Không bao giờ huấn luyện mô hình từ đầu vì tốn **hàng trăm triệu đô la**.
Fine-tuning giúp mô hình trở nên **tốt hơn cho tác vụ cụ thể** mà nó đang được điều chỉnh.

Tìm hiểu 3 bước để fine-tuning với OpenAI.

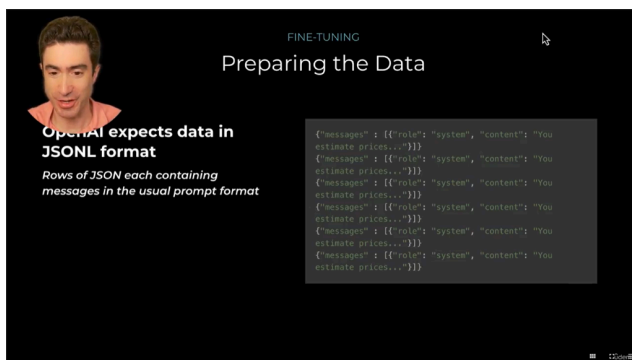


Cần có dữ liệu huấn luyện để tinh chỉnh mô hình GPT-4.
Dữ liệu phải được định dạng theo JSON Lines (JSONL), khác một chút so với JSON thông thường.
Dữ liệu này sau đó được tải lên OpenAI.

Chạy quá trình huấn luyện (fine-tuning) và theo dõi **training loss**.
 Training loss giảm có nghĩa là mô hình đang học tốt hơn.
 Cần theo dõi **validation loss** để tránh overfitting (mô hình học quá kỹ dữ liệu huấn luyện mà không tổng quát hóa tốt trên dữ liệu mới).

Epoch là số lần mô hình chạy hết tập dữ liệu huấn luyện.
Do có nhiều dữ liệu, chỉ cần chạy một epoch thay vì lặp lại nhiều lần.
Vì dữ liệu luôn mới, training loss có thể được sử dụng tương đương với validation loss.

Sau khi huấn luyện, đánh giá kết quả.
Dựa trên kết quả, điều chỉnh lại dữ liệu và mô hình để tiếp tục cải thiện.



JSON là tập hợp các dòng dữ liệu JSON riêng biệt, không được gói trong danh sách ([]).
 Mỗi dòng là một đối tượng JSON độc lập, bắt đầu và kết thúc bằng dấu { }.
 Khác với JSON thông thường (dữ liệu JSON thông thường có thể là một danh sách chứa nhiều đối tượng JSON).

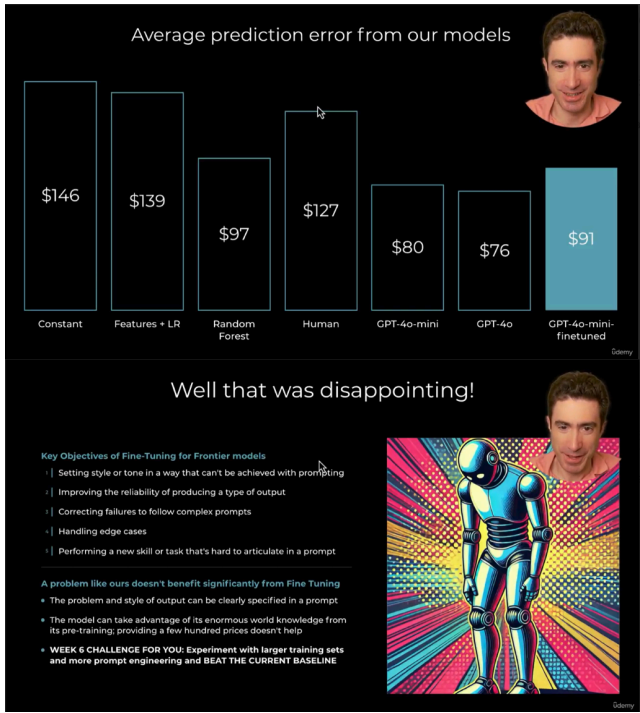
Mỗi dòng JSON chứa một thuộc tính chính: "messages".
 "messages" là danh sách các từ điển (list of dictionaries), mỗi từ điển chứa hai trường:
 "role": Vai trò trong cuộc trò chuyện (ví dụ: "system", "user", "assistant").
 "content": Nội dung tin nhắn tương ứng.

Cấu trúc này phản ánh một cuộc hội thoại và là định dạng mà mô hình GPT-4 sử dụng để học hỏi.

Tiến hành tạo tập dữ liệu theo định dạng JSONL.

Sau khi chuẩn bị xong, sử dụng **Jupyter Lab** để chạy quá trình huấn luyện.

Lần đầu tiên sẽ thực hiện huấn luyện một mô hình tiên phong ("frontier model").



Đánh giá kết quả fine-tuning

- So sánh lỗi dự đoán giữa các phương pháp khác nhau, từ mô hình truyền thống đến GPT-4 Mini và GPT-4.
- GPT-4 Mini ban đầu đạt lỗi 80, sau fine-tuning lại tăng lên 91, cho thấy kết quả fine-tuning không cải thiện.
- Có thể do fine-tuning chưa được tối ưu hoặc do hiện tượng "quên lãng thảm khốc" (catastrophic forgetting).

Mục tiêu của fine-tuning trên mô hình mạnh (frontier model)

- Fine-tuning thường dùng để cải thiện mô hình nhỏ hơn để đạt hiệu suất tốt hơn.
- Đối với mô hình lớn như GPT-4, fine-tuning có thể không hiệu quả nếu mục tiêu không rõ ràng.

Các lý do chính để fine-tune một mô hình lớn (theo OpenAI)

- Tạo phong cách phản hồi:** Điều chỉnh giọng điệu (ví dụ, thêm châm biếm).
- Tạo đầu ra theo định dạng cố định:** Cần kết quả có cấu trúc cụ thể.
- Sửa lỗi mô hình với các yêu cầu phức tạp:** Khi mô hình không hiểu hoặc thực hiện sai yêu cầu khó.
- Xử lý trường hợp hiếm gặp:** Sửa các lỗi xảy ra trong một số tình huống đặc biệt.
- Thực hiện nhiệm vụ mới khó mô tả bằng prompt:** Khi không thể đơn giản hóa yêu cầu chỉ bằng prompt.

Tầm quan trọng của prompt engineering

- OpenAI nhấn mạnh rằng, trước khi fine-tune, cần tối ưu prompt trước.
- GPT-4 Mini đã hiểu tốt yêu cầu từ prompt và tạo đầu ra đúng định dạng.
- Do đó, fine-tuning có thể không cần thiết nếu prompt đã hiệu quả.

Thách thức tiếp theo

- Tối ưu thêm siêu tham số (hyperparameter tuning) để cải thiện fine-tuning.
- Thử nghiệm với tập dữ liệu lớn hơn (dù OpenAI không khuyến nghị dùng tập quá lớn).
- Xác định xem fine-tuning có thật sự mang lại lợi ích so với chỉ cải thiện prompt hay không.

Kết quả thử nghiệm Fine-tuning

- Khi chạy mô hình dựa trên giá trị trung bình, sai số trung bình là 146.
- Machine learning truyền thống giảm sai số xuống 139, Random Forest còn 97.
- GPT-4 Mini lần đầu tiên đạt 80, GPT-4 đạt 76, nhưng lần fine-tuning gần đây lại giảm xuống 91.
- Điều này cho thấy fine-tuning có thể không luôn cải thiện kết quả trên mô hình lớn.

Mục tiêu của Fine-tuning trên mô hình lớn

- Điều chỉnh phong cách hoặc giọng điệu của phản hồi (ví dụ: thêm tính châm biếm).
- Định dạng đầu ra theo một cấu trúc cụ thể.
- Cải thiện khả năng hiểu những lời nhắc phức tạp.
- Xử lý các trường hợp đặc biệt mà mô hình thường mắc lỗi.
- Dạy mô hình thực hiện một nhiệm vụ mới không thể dễ dàng mô tả qua prompt.

Hạn chế của Fine-tuning trên mô hình lớn

- GPT-4 đã được huấn luyện trên lượng dữ liệu khổng lồ, thêm một lượng nhỏ dữ liệu mới ít có khả năng cải thiện đáng kể.
- Fine-tuning có thể gây ra "quên thảm họa" (catastrophic forgetting), làm giảm khả năng tổng quát của mô hình.
- OpenAI nhấn mạnh rằng việc tối ưu prompt thường hiệu quả hơn là fine-tuning.

Thử nghiệm tiếp theo

- Điều chỉnh siêu tham số (hyperparameter tuning) như tập dữ liệu lớn hơn (1000-2000 mẫu), số epoch, và các tham số khác trên OpenAI.
- Thử nghiệm nhiều cách viết prompt khác nhau để xem có cải thiện được kết quả hay không.
- Mục tiêu là cải thiện sai số xuống thấp hơn mức 76 đã từng đạt được.

Kết thúc tuần 6 - Chuẩn bị tuần 7

- Người học đã đi được 75% chặng đường để trở thành kỹ sư LLM.
- Đã học cách sử dụng mô hình tiên tiến, Hugging Face, LangChain, và fine-tuning mô hình frontier.
- Tuần tới sẽ tập trung vào fine-tuning mô hình mã nguồn mở với kỹ thuật LoRA, khác biệt so với fine-tuning mô hình lớn như GPT-4.
- Mục tiêu cuối cùng là tìm cách tối ưu mô hình mã nguồn mở để cạnh tranh với GPT-4.

=> Bài học rút ra:

- Fine-tuning không phải lúc nào cũng giúp cải thiện mô hình lớn, và tối ưu prompt là bước đầu quan trọng.
- Để fine-tuning hiệu quả trên mô hình nhỏ, cần hiểu rõ cách chọn mô hình gốc, tập dữ liệu, và kỹ thuật như LoRA.
- Kết quả tốt nhất đạt được qua thử nghiệm và tối ưu siêu tham số.



